Feature Engineering

Agenda

- Introduction to Feature Engineering
- Feature Engineering Methods
- Transforming Variables
  - Log Transformation
  - Square Root Transformation
  - Box-Cox Transformation
- Features Scaling
- Label Encoding
- One Hot Encoding
- Hashing
  - Hashlib Module
- Grouping Operations

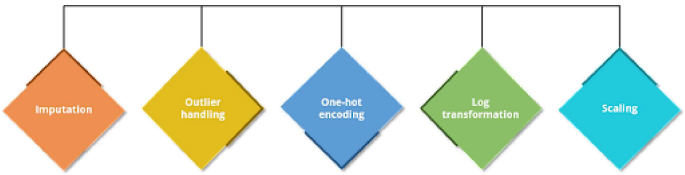
1. Introduction to Feature Engineering

It refers to the process of selecting, modifying, or creating new features (variables) from the raw data to improve the performance of machine learning models.

- It involves transforming the data into a more suitable format, making it easier for models to learn patterns and make accurate predictions.
- It is a critical step in the data preprocessing pipeline and plays a key role in the success of machine learning projects.

2. Feature Engineering Methods

They introduce the concept of creating new features through mathematical operations, transformations, or combining existing variables.



Note: The **Data Wrangling** lesson extensively addresses various feature engineering methods, including outlier handling, imputation, and data cleaning. Any aspects not covered in that lesson but deemed essential for feature engineering are comprehensively discussed here.

```
import pandas as pd
import numpy as np
df= pd.read_csv("HousePrices.csv")
# Example: Creating a new feature 'total_rooms' by adding bedrooms and bathrooms
df['total_rooms'] = df['bedrooms'] + df['bathrooms']
```

3. Transforming Variables

Transforming variables is a crucial aspect of feature engineering that involves modifying the scale, distribution, or nature of variables to meet certain assumptions or to make them more suitable for analysis or modeling.

- Here are a few common techniques for transforming variables:

1. Log transformation
2. Square root transformation
3. Box-cox transformation

3.1 Log Transformation

Log transformation is useful for handling skewed data or reducing the impact of outliers. It applies the natural logarithm to the variable values and makes highly skewed distributions less skewed.

```
# Logarithmic transformation of the 'price' column
df['log_price'] = np.log(df['price'])
```

```
/usr/local/lib/python3.10/dist-packages/pandas/core/arraylike.py:396: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

3.2 Square Root Transformation

Square root transformation, like log transformation, effectively stabilizes variance and addresses skewed distributions. Although it's gentler than log transformation, it achieves the same objective.

```
# Square root transforming the 'price' variable
df['SquareRoot_price'] = np.sqrt(df['price'])

# Displaying the DataFrame with the new feature
print("DataFrame with square root transformed 'price':")
print(df[['price', 'SquareRoot_price']])
```

```
DataFrame with square root transformed 'price':
             price  SquareRoot_price
0     3.130000e+05        559.464029
1     2.384000e+06       1544.020725
2     3.420000e+05        584.807661
3     4.200000e+05        648.074070
4     5.500000e+05        741.619849
...            ...               ...
4595  3.081667e+05        555.127613
4596  5.343333e+05        730.981076
4597  4.169042e+05        645.681165
4598  2.034000e+05        450.998891
4599  2.206000e+05        469.680743

[4600 rows x 2 columns]
```

3.3 Box-Cox Transformation

The box-cox transformation is a family of power transformations that includes log and square root transformations.

- It can handle a broader range of data distributions.
- Ensuring positive data is crucial for the Box-Cox transformation because it involves taking the logarithm, which is undefined for zero or negative values. Adding a constant helps avoid mathematical errors and ensures the transformation can be applied effectively.

```
from scipy.stats import boxcox

# Applying Box-Cox transformation to 'sales' variable
df['BoxCox_Price'], _ = boxcox(df['sqft_living'])

# Displaying the DataFrame with the Box-Cox transformed 'sales' variable
print("DataFrame with box-cox transformed price:")
print(df[['sqft_living', 'BoxCox_Price']])
```

```
DataFrame with box-cox transformed price:
      sqft_living  BoxCox_Price
0            1340      8.270696
1            3650      9.609817
2            1930      8.752434
3            2000      8.799831
4            1940      8.759306
...           ...           ...
4595         1510      8.427673
4596         1460      8.383346
4597         3010      9.348243
4598         2090      8.858477
4599         1490      8.410115
```

```
[4600 rows x 2 columns]
```

4. Feature Scaling

Feature scaling is a technique used in machine learning and data preprocessing to standardize or normalize the range of independent variables or features of a dataset.

- Min-max scaling transforms data to a specific range, typically between 0 and 1, preserving the relative differences between values. This normalization technique is ideal for datasets with known bounds, ensuring that all values are rescaled proportionally to fit within the specified range.
- Standard scaling is preferable for normally distributed data to maintain mean-centeredness and consistent standard deviations.

Label encoding

Food name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One hot encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

```
from sklearn.preprocessing import MinMaxScaler

# Scaling numeric features using min-max scaling
scaler = MinMaxScaler()
df[['sqft_living', 'sqft_lot']] = scaler.fit_transform(df[['sqft_living', 'sqft_lot']])
print(df)
```

```
                     date         price  bedrooms  bathrooms  sqft_living  \
0     2014-05-02 00:00:00  3.130000e+05       3.0       1.50     0.073652
1     2014-05-02 00:00:00  2.384000e+06       5.0       2.50     0.249051
2     2014-05-02 00:00:00  3.420000e+05       3.0       2.00     0.118451
3     2014-05-02 00:00:00  4.200000e+05       3.0       2.25     0.123766
4     2014-05-02 00:00:00  5.500000e+05       4.0       2.50     0.119210
...                   ...           ...       ...        ...          ...
4595  2014-07-09 00:00:00  3.081667e+05       3.0       1.75     0.086560
4596  2014-07-09 00:00:00  5.343333e+05       3.0       2.50     0.082764
4597  2014-07-09 00:00:00  4.169042e+05       3.0       2.50     0.200456
4598  2014-07-10 00:00:00  2.034000e+05       4.0       2.00     0.130600
4599  2014-07-10 00:00:00  2.206000e+05       3.0       2.50     0.085042

      sqft_lot  floors  waterfront  view  condition  ...  yr_built  \
0     0.006775     1.5           0     0          3  ...      1955
1     0.007835     2.0           0     4          5  ...      1921
2     0.010534     1.0           0     0          4  ...      1966
3     0.006885     1.0           0     0          4  ...      1963
4     0.009186     1.0           0     0          4  ...      1976
...        ...     ...         ...   ...        ...  ...       ...
4595  0.005330     1.0           0     0          4  ...      1954
4596  0.006460     2.0           0     0          3  ...      1983
4597  0.005939     2.0           0     0          3  ...      2009
4598  0.005581     1.0           0     0          3  ...      1974
4599  0.006952     2.0           0     0          4  ...      1990

      yr_renovated                    street       city  statezip country  \
0             2005      18810 Densmore Ave N  Shoreline  WA 98133     USA
1                0           709 W Blaine St    Seattle  WA 98119     USA
2                0  26206-26214 143rd Ave SE       Kent  WA 98042     USA
3                0           857 170th Pl NE   Bellevue  WA 98008     USA
4             1992         9105 170th Ave NE    Redmond  WA 98052     USA
...            ...                       ...        ...       ...     ...
4595          1979            501 N 143rd St    Seattle  WA 98133     USA
4596          2009          14855 SE 10th Pl   Bellevue  WA 98007     USA
4597             0          759 Ilwaco Pl NE     Renton  WA 98059     USA
4598             0         5148 S Creston St    Seattle  WA 98178     USA
4599             0         18717 SE 258th St  Covington  WA 98042     USA
```

```
     total_rooms  log_price  SquareRoot_price  BoxCox_Price
0           4.50  12.653958        559.464029      8.270696
1           7.50  14.684290       1544.020725      9.609817
2           5.00  12.742566        584.807661      8.752434
3           5.25  12.948010        648.074070      8.799831
4           6.50  13.217674        741.619849      8.759306
...          ...        ...               ...           ...
4595        4.75  12.638396        555.127613      8.427673
```

5. Label Encoding

Label encoding is a technique used to convert categorical labels into a numeric format, making it suitable for machine learning algorithms that require numerical input.

- In label encoding, each unique category is assigned an integer value.
- This is particularly useful when dealing with ordinal categorical data, where the order of categories matters.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Sample DataFrame
data = {'size': ['small', 'medium', 'large', 'medium', 'small']}
df1 = pd.DataFrame(data)

# Before label encoding
print("Original DataFrame:")
print(df1)

# Apply label encoding
label_encoder = LabelEncoder()
df1['size_encoded'] = label_encoder.fit_transform(df1['size'])

# After label encoding
print("\nDataFrame after label encoding:")
print(df1)
```

```
Original DataFrame:
     size
0   small
1  medium
2   large
3  medium
4   small

DataFrame after label encoding:
     size  size_encoded
0   small             2
1  medium             1
2   large             0
3  medium             1
4   small             2
```

```
# Demonstrating label encoding using csv file
from sklearn.preprocessing import LabelEncoder

# Label encoding for the 'city' column
label_encoder = LabelEncoder()
df['city_encoded'] = label_encoder.fit_transform(df['city'])
print(df)
```

```
                     date         price  bedrooms  bathrooms  sqft_living  \
0     2014-05-02 00:00:00  3.130000e+05       3.0       1.50     0.073652
1     2014-05-02 00:00:00  2.384000e+06       5.0       2.50     0.249051
2     2014-05-02 00:00:00  3.420000e+05       3.0       2.00     0.118451
3     2014-05-02 00:00:00  4.200000e+05       3.0       2.25     0.123766
4     2014-05-02 00:00:00  5.500000e+05       4.0       2.50     0.119210
...                   ...           ...       ...        ...          ...
4595  2014-07-09 00:00:00  3.081667e+05       3.0       1.75     0.086560
4596  2014-07-09 00:00:00  5.343333e+05       3.0       2.50     0.082764
4597  2014-07-09 00:00:00  4.169042e+05       3.0       2.50     0.200456
4598  2014-07-10 00:00:00  2.034000e+05       4.0       2.00     0.130600
4599  2014-07-10 00:00:00  2.206000e+05       3.0       2.50     0.085042

      sqft_lot  floors  waterfront  view  condition  ...  yr_renovated  \
0     0.006775     1.5           0     0          3  ...          2005
```

```
1     0.007835     2.0           0     4          5  ...             0
2     0.010534     1.0           0     0          4  ...             0
3     0.006885     1.0           0     0          4  ...             0
4     0.009186     1.0           0     0          4  ...          1992
...        ...     ...         ...   ...        ...  ...           ...
4595  0.005330     1.0           0     0          4  ...          1979
4596  0.006460     2.0           0     0          3  ...          2009
4597  0.005939     2.0           0     0          3  ...             0
4598  0.005581     1.0           0     0          3  ...             0
4599  0.006952     2.0           0     0          4  ...             0

                        street       city  statezip country total_rooms  \
0         18810 Densmore Ave N  Shoreline  WA 98133     USA        4.50
1              709 W Blaine St    Seattle  WA 98119     USA        7.50
2     26206-26214 143rd Ave SE       Kent  WA 98042     USA        5.00
3              857 170th Pl NE   Bellevue  WA 98008     USA        5.25
4            9105 170th Ave NE    Redmond  WA 98052     USA        6.50
...                        ...        ...       ...     ...         ...
4595            501 N 143rd St    Seattle  WA 98133     USA        4.75
4596          14855 SE 10th Pl   Bellevue  WA 98007     USA        5.50
4597          759 Ilwaco Pl NE     Renton  WA 98059     USA        5.50
4598         5148 S Creston St    Seattle  WA 98178     USA        6.00
4599         18717 SE 258th St  Covington  WA 98042     USA        5.50

      log_price SquareRoot_price  BoxCox_Price  city_encoded
0     12.653958       559.464029      8.270696            36
1     14.684290      1544.020725      9.609817            35
2     12.742566       584.807661      8.752434            18
3     12.948010       648.074070      8.799831             3
4     13.217674       741.619849      8.759306            31
...         ...              ...           ...           ...
4595  12.638396       555.127613      8.427673            35
4596  13.188775       730.981076      8.383346             3
```

6. One-Hot Encoding

One-hot encoding is a technique to represent categorical variables as binary vectors.

- It is particularly useful when dealing with nominal categorical data, where there is no inherent order among categories.
- In one-hot encoding, each unique category is transformed into a binary column, and only one column in each set of binary columns is *hot* (or 1) to indicate the presence of that category.
- It increases dataset dimensionality, facilitating categorical data representation. However, it can lead to increased complexity and computational overhead.

```
import pandas as pd

# Sample DataFrame
data = {'color': ['red', 'blue', 'green', 'red', 'green']}
df2 = pd.DataFrame(data)

# Before one-hot encoding
print("Original DataFrame:")
print(df2)

# Apply one-hot encoding
df2_encoded = pd.get_dummies(df2, columns=['color'], prefix='color')

# After one-hot encoding
print("\nDataFrame after one-hot encoding:")
print(df2_encoded)
```

```
Original DataFrame:
   color
0    red
1   blue
2  green
3    red
4  green

DataFrame after one-hot encoding:
   color_blue  color_green  color_red
0       False        False       True
1        True        False      False
2       False         True      False
```

```
3       False        False       True
4       False         True      False
```

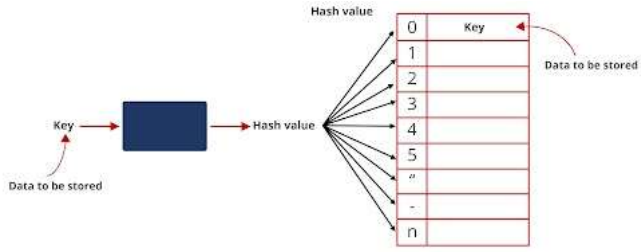
```
# Demonstrating one-hot encoding using csv file
# One-Hot Encoding for the 'view' column
df_encode = pd.get_dummies(df, columns=['price'], prefix='price')

# After one-hot encoding
print("\nDataFrame after one-hot encoding:")
print(df_encode)
```

```
DataFrame after one-hot encoding:
                     date  bedrooms  bathrooms  sqft_living  sqft_lot  floors  \
0     2014-05-02 00:00:00       3.0       1.50     0.073652  0.006775     1.5
1     2014-05-02 00:00:00       5.0       2.50     0.249051  0.007835     2.0
2     2014-05-02 00:00:00       3.0       2.00     0.118451  0.010534     1.0
3     2014-05-02 00:00:00       3.0       2.25     0.123766  0.006885     1.0
4     2014-05-02 00:00:00       4.0       2.50     0.119210  0.009186     1.0
...                   ...       ...        ...          ...       ...     ...
4595  2014-07-09 00:00:00       3.0       1.75     0.086560  0.005330     1.0
4596  2014-07-09 00:00:00       3.0       2.50     0.082764  0.006460     2.0
4597  2014-07-09 00:00:00       3.0       2.50     0.200456  0.005939     2.0
4598  2014-07-10 00:00:00       4.0       2.00     0.130600  0.005581     1.0
4599  2014-07-10 00:00:00       3.0       2.50     0.085042  0.006952     2.0

      waterfront  view  condition  sqft_above  ...  price_3000000.0  \
0              0     0          3        1340  ...            False
1              0     4          5        3370  ...            False
2              0     0          4        1930  ...            False
3              0     0          4        1000  ...            False
4              0     0          4        1140  ...            False
...          ...   ...        ...         ...  ...              ...
4595           0     0          4        1510  ...            False
4596           0     0          3        1460  ...            False
4597           0     0          3        3010  ...            False
4598           0     0          3        1070  ...            False
4599           0     0          4        1490  ...            False

      price_3100000.0  price_3200000.0 price_3710000.0 price_3800000.0  \
0               False            False           False           False
1               False            False           False           False
2               False            False           False           False
3               False            False           False           False
4               False            False           False           False
...               ...              ...             ...             ...
4595            False            False           False           False
4596            False            False           False           False
4597            False            False           False           False
4598            False            False           False           False
4599            False            False           False           False

     price_4489000.0 price_4668000.0  price_7062500.0  price_12899000.0  \
0              False           False            False             False
1              False           False            False             False
2              False           False            False             False
3              False           False            False             False
4              False           False            False             False
...              ...             ...              ...               ...
```

7. Hashing

It is a technique to convert input data (of variable length) into a fixed-length string of characters, typically a hash code.

- The hash function takes an input (or message) and returns a fixed-size string of characters, which is typically a hexadecimal number.
- It is commonly used for indexing data structures, checking data integrity, and hashing passwords.




```
# Example of hashing in Python
data = "Hello, Hashing!"

# Using the hash() function
hash_value = hash(data)

print(f"Original data: {data}")
print(f"Hash value: {hash_value}")
```

```
Original data: Hello, Hashing!
Hash value: -4999438547603160871
```

```
# Demonstrating hashing using csv file
# Hashing for the 'street' column
df['street_hashed'] = df['street'].apply(hash)
df
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	...	street
0	2014-05-02 00:00:00	3.130000e+05	3.0	1.50	0.073652	0.006775	1.5	0	0	3	...	18810 Densmore Ave N
1	2014-05-02 00:00:00	2.384000e+06	5.0	2.50	0.249051	0.007835	2.0	0	4	5	...	709 W Blaine St
2	2014-05-02 00:00:00	3.420000e+05	3.0	2.00	0.118451	0.010534	1.0	0	0	4	...	26206-26214 143rd Ave SE
3	2014-05-02 00:00:00	4.200000e+05	3.0	2.25	0.123766	0.006885	1.0	0	0	4	...	857 170th Pl NE
4	2014-05-02 00:00:00	5.500000e+05	4.0	2.50	0.119210	0.009186	1.0	0	0	4	...	9105 170th Ave NE
...	...	...	...	...	...	...	...	...	...	...	...	...
4595	2014-07-09 00:00:00	3.081667e+05	3.0	1.75	0.086560	0.005330	1.0	0	0	4	...	501 N 143rd St
4596	2014-07-09 00:00:00	5.343333e+05	3.0	2.50	0.082764	0.006460	2.0	0	0	3	...	14855 SE 10th Pl
4597	2014-07-09 00:00:00	4.169042e+05	3.0	2.50	0.200456	0.005939	2.0	0	0	3	...	759 Ilwaco Pl NE
4598	2014-07-10 00:00:00	2.034000e+05	4.0	2.00	0.130600	0.005581	1.0	0	0	3	...	5148 S Creston St
4599	2014-07-10 00:00:00	2.206000e+05	3.0	2.50	0.085042	0.006952	2.0	0	0	4	...	18717 SE 258th St

4600 rows × 24 columns

7.1 Hashlib Module

The hashlib module in Python is used for generating hash values. It offers interfaces to different cryptographic hash algorithms like MD5, SHA-1, SHA-256, SHA-384, and SHA-512.

- It enables the efficient use of hash functions, ensuring secure computations.
- It provides reliability for hash-related operations.

- It is widey used for cryptographic operations, data integrity, and password hashing.
- It ensures convenience and robustness.

Cryptographic hash algorithms vary in hash size and security levels.

- For tasks where security is not a critical concern, you can opt for MD5 or SHA-1. However, it's important to note that both algorithms are deprecated due to vulnerabilities.
- For security-sensitive applications, it's advisable to prioritize SHA-256, SHA-384, or SHA-512 due to their stronger security and larger hash sizes.

```
# Example of hashlib module in Python
import hashlib

# Input data
data = b'Hello, world!'
print(f"Original data: {data.decode()} \n")

# Calculate MD5 hash
md5_hash = hashlib.md5(data).hexdigest()
print("MD5 Hash:", md5_hash)

# Calculate SHA-1 hash
sha1_hash = hashlib.sha1(data).hexdigest()
print("SHA-1 Hash:", sha1_hash)

# Calculate SHA-256 hash
sha256_hash = hashlib.sha256(data).hexdigest()
print("SHA-256 Hash:", sha256_hash)

# Calculate SHA-384 hash
sha384_hash = hashlib.sha384(data).hexdigest()
print("SHA-384 Hash:", sha384_hash)

# Calculate SHA-512 hash
sha512_hash = hashlib.sha512(data).hexdigest()
print("SHA-512 Hash:", sha512_hash)
```

```
Original data: Hello, world!

MD5 Hash: 6cd3556deb0da54bca060b4c39479839
SHA-1 Hash: 943a702d06f34599aee1f8da8ef9f7296031d699
SHA-256 Hash: 315f5bdb76d078c43b8ac0064e4a0164612b1fce77c869345bfc94c75894edd3
SHA-384 Hash: 55bc556b0d2fe0fce582ba5fe07baafff035653638c7ac0d5494c2a64c0bea1cc57331c7c12a45cdbca7f4c34a089eeb
SHA-512 Hash: c1527cd893c124773d811911970c8fe6e857d6df5dc9226bd8a160614c0cd963a4ddea2b94bb7d36021ef9d865d5cea294a82dd49a0bb2
```

In this example, the `hashlib` module is imported and input data is provided in bytes format. Hash values are then computed using the md5(), sha1(), sha256(), sha384(), and sha512() functions, and their hexadecimal representations are obtained using hexdigest().

```
# Demonstrating MD5 hash function using csv file for the 'street' column

street_column = df['street']
hashed_streets = street_column.apply(lambda x: hashlib.md5(x.encode()).hexdigest())

# Replace the original street values with hash values
df['hashed_street'] = hashed_streets

# Optionally, write the updated DataFrame back to a CSV file
df.to_csv('hashed_file.csv', index=False)

df
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	...	city
0	2014-05-02 00:00:00	3.130000e+05	3.0	1.50	0.073652	0.006775	1.5	0	0	3	...	Shoreline
1	2014-05-02 00:00:00	2.384000e+06	5.0	2.50	0.249051	0.007835	2.0	0	4	5	...	Seattle
2	2014-05-02 00:00:00	3.420000e+05	3.0	2.00	0.118451	0.010534	1.0	0	0	4	...	Kent
3	2014-05-02 00:00:00	4.200000e+05	3.0	2.25	0.123766	0.006885	1.0	0	0	4	...	Bellevue
4	2014-05-02 00:00:00	5.500000e+05	4.0	2.50	0.119210	0.009186	1.0	0	0	4	...	Redmond
...	...	...	...	...	...	...	...	...	...	...	...	...
4595	2014-07-09 00:00:00	3.081667e+05	3.0	1.75	0.086560	0.005330	1.0	0	0	4	...	Seattle
4596	2014-07-09 00:00:00	5.343333e+05	3.0	2.50	0.082764	0.006460	2.0	0	0	3	...	Bellevue
4597	2014-07-09 00:00:00	4.169042e+05	3.0	2.50	0.200456	0.005939	2.0	0	0	3	...	Renton
4598	2014-07-10 00:00:00	2.034000e+05	4.0	2.00	0.130600	0.005581	1.0	0	0	3	...	Seattle
4599	2014-07-10 00:00:00	2.206000e+05	3.0	2.50	0.085042	0.006952	2.0	0	0	4	...	Covington

4600 rows × 25 columns

8. Grouping Operations

Grouping operations involve splitting a dataset into groups based on some criteria, applying a function to each group independently, and then combining the results.

- This is a crucial step in data analysis and manipulation, allowing for insights into the data at a more granular level.
- Grouping operations are commonly combined with aggregate functions to summarize data within each group.

```
import pandas as pd

# Sample DataFrame
data = {'Category': ['Electronics', 'Clothing', 'Electronics', 'Clothing', 'Electronics'],
        'Revenue': [500, 300, 700, 400, 600]}

df3 = pd.DataFrame(data)

# Grouping by 'Category' and calculating total revenue for each category
grouped_df3 = df3.groupby('Category')['Revenue'].sum().reset_index()

print("Original DataFrame:")
print(df3)
print("\nGrouped DataFrame with total revenue:")
print(grouped_df3)
```

```
Original DataFrame:
      Category  Revenue
0  Electronics      500
1     Clothing      300
2  Electronics      700
3     Clothing      400
4  Electronics      600

Grouped DataFrame with total revenue:
      Category  Revenue
0     Clothing      700
1  Electronics     1800
```

```
# Grouping by 'city' and calculating the average price
df_grouped = df.groupby('city')['price'].mean().reset_index()
print(df_grouped)
```

```
                   city         price
0                Algona  2.072880e+05
1                Auburn  2.993404e+05
2    Beaux Arts Village  7.450000e+05
3              Bellevue  8.471807e+05
4         Black Diamond  3.396056e+05
5               Bothell  4.814419e+05
6                Burien  3.489472e+05
7             Carnation  5.087520e+05
8            Clyde Hill  1.321945e+06
9             Covington  2.962304e+05
10           Des Moines  3.049925e+05
11               Duvall  4.039941e+05
12             Enumclaw  3.076146e+05
13            Fall City  6.926818e+05
14          Federal Way  2.898877e+05
15  Inglewood-Finn Hill  4.250000e+05
16             Issaquah  5.961637e+05
17              Kenmore  4.474940e+05
18                 Kent  4.394924e+05
19             Kirkland  6.515836e+05
20     Lake Forest Park  4.484750e+05
21         Maple Valley  3.364749e+05
22               Medina  2.046559e+06
23        Mercer Island  1.123818e+06
24               Milton  2.850000e+05
25            Newcastle  6.660467e+05
26        Normandy Park  5.067931e+05
27           North Bend  3.995657e+05
28              Pacific  2.252333e+05
29              Preston  5.624500e+05
30           Ravensdale  5.140714e+05
31              Redmond  6.676495e+05
32               Renton  3.770410e+05
33            Sammamish  6.869176e+05
34               SeaTac  2.452906e+05
35              Seattle  5.798375e+05
36            Shoreline  4.203924e+05
37            Skykomish  2.330000e+05
38           Snoqualmie  5.363053e+05
39      Snoqualmie Pass  5.250000e+05
40              Tukwila  3.082901e+05
41               Vashon  4.725569e+05
42          Woodinville  6.095650e+05
43         Yarrow Point  1.194838e+06
```

```
# Grouping by 'city' and calculating the maximum price
df_max_price = df.groupby('city')['price'].max().reset_index()
print(df_max_price)
```

```
                   city       price
0                Algona    262000.0
1                Auburn    900000.0
2    Beaux Arts Village    745000.0
3              Bellevue   7062500.0
4         Black Diamond    650000.0
5               Bothell    749995.0
6                Burien   1035000.0
7             Carnation   1680000.0
8            Clyde Hill   3800000.0
9             Covington   2199900.0
10           Des Moines    950000.0
11               Duvall    955000.0
12             Enumclaw    687000.0
13            Fall City   1600000.0
14          Federal Way    819000.0
15  Inglewood-Finn Hill    425000.0
16             Issaquah   2238888.0
17              Kenmore   1120000.0
```

```
18                Kent  26590000.0
19            Kirkland   3000000.0
20    Lake Forest Park    790000.0
21        Maple Valley    725000.0
22               Medina   3710000.0
23       Mercer Island   4668000.0
24              Milton    320000.0
25           Newcastle   1200000.0
26       Normandy Park   1309500.0
27          North Bend    845000.0
28             Pacific    260000.0
29             Preston    685000.0
30          Ravensdale    792500.0
31             Redmond   2280000.0
32              Renton   1135250.0
33           Sammamish   2400000.0
34              SeaTac    489000.0
35             Seattle  12899000.0
36           Shoreline   2005000.0
37           Skykomish    415000.0
38          Snoqualmie   1149000.0
39     Snoqualmie Pass    525000.0
40             Tukwila   2110000.0
41              Vashon    849900.0
42         Woodinville   1820000.0
43        Yarrow Point   1901000.0
```

```
# Grouping by 'city' and calculating the minimum price
df_min_price = df.groupby('city')['price'].min().reset_index()
print(df_min_price)
```

```
                   city          price
0                Algona  100000.000000
1                Auburn       0.000000
2    Beaux Arts Village  745000.000000
3              Bellevue       0.000000
4         Black Diamond       0.000000
5               Bothell  347000.000000
6                Burien       0.000000
7             Carnation   80000.000000
8            Clyde Hill       0.000000
9             Covington       0.000000
10           Des Moines  140000.000000
11               Duvall  117833.333333
12             Enumclaw       0.000000
13            Fall City  275000.000000
14          Federal Way       0.000000
15  Inglewood-Finn Hill  425000.000000
16             Issaquah       0.000000
17              Kenmore       0.000000
18                 Kent       0.000000
19             Kirkland   90000.000000
20     Lake Forest Park       0.000000
21         Maple Valley       0.000000
22               Medina  188000.000000
23        Mercer Island       0.000000
24               Milton  250000.000000
25            Newcastle  339900.000000
26        Normandy Park       0.000000
27           North Bend  240000.000000
28              Pacific  174000.000000
29              Preston  439900.000000
30           Ravensdale  225000.000000
31              Redmond  170000.000000
32               Renton       0.000000
33            Sammamish       0.000000
34               SeaTac       0.000000
35              Seattle       0.000000
36            Shoreline  176225.000000
37            Skykomish  134000.000000
38           Snoqualmie       0.000000
39      Snoqualmie Pass  525000.000000
40              Tukwila    7800.000000
41               Vashon  160000.000000
```

```
42          Woodinville       0.000000
43         Yarrow Point   84350.000000
```

Assisted Practice

Problem Statement:

A botanical research team is conducting a comprehensive analysis of iris flowers, aiming to derive valuable insights from their characteristics. The team wants to explore feature engineering techniques to understand and visualize the relationships within the Iris dataset.

Steps to perform:

- Understand the Dataset: Get familiar with the Iris dataset and its features
- Engineer Features: Create new features like the sepal and petal areas
- Transform Variables: If the features are not normally distributed, apply transformations